Data and Artificial Intelligence
Cyber Shujaa Program

Week 7 Assignment
Regression Models

Student Name: Rodney Roy Gitonga
Student ID: CS-DA03-26025

Contents

Introduction 2
Tasks Completed 2
Step 1: Data Exploration 2
To begin the analysis, the dataset was loaded and examined using the df.describe() function to understand its underlying statistical distribution (such as the mean, minimum, and maximum values for the home areas and prices). An initial scatter plot was also generated to visualize the relationship between the independent variable (area) and the dependent variable (price). As demonstrated in the chart, the data shows a strong positive correlation; this means that as the square footage area of a house increases, its price reliably increases as well. 2
Step 2: Data Preparation 2
Before building the model, the data was separated into features (X) and target labels (y). Using Scikit-learn's train_test_split function, the data was divided into an 80/20 split—meaning 80% of the data was allocated for training the model, and 20% was set aside for testing. This split is standard practice because it ensures the model can be evaluated on unseen data, allowing us to accurately measure how well it generalizes to new inputs rather than just memorizing the data it was trained on. 2
Step 3: Model Training 3
Next, a Linear Regression model was instantiated and fitted to the training data. During this training phase, the algorithm calculated the optimal line of best fit by determining the mathematical relationship between the area and the price. Based on the training data provided, the model calculated a coefficient (slope) of 128.27 and an intercept of 211,542.06. 3
Step 4: Evaluation 3
Once the model was trained, its predictive accuracy was tested on 20% of the data unseen. The model's performance was evaluated using standard regression metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and the R^2 score. MAE provides the average absolute magnitude of prediction errors. Conversely, MSE and RMSE square

the errors before averaging them. Because of this squaring step, RMSE penalizes larger errors much more heavily than MAE, making it a highly effective metric for identifying whether the model's predictions contain any extreme outliers. 3
Step 5: Visualization 4
Conclusion 4

Introduction

This project builds a Simple Linear Regression model to predict the price of a home based entirely on its square footage area.

The dataset contains housing records with areas ranging from 2600 to 4000 square feet.

The corresponding home prices in the dataset range from 550,000 to 725,000.The mathematical foundation of the model is $Y = mX + b$, where Y is the predicted price and X is the area.

Tasks Completed

Step 1: Data Exploration

To begin the analysis, the dataset was loaded and examined using the df.describe() function to understand its underlying statistical distribution (such as the mean, minimum, and maximum values for the home areas and prices). An initial scatter plot was also generated to visualize the relationship between the independent variable (area) and the dependent variable (price). As demonstrated in the chart, the data shows a strong positive correlation; this means that as the square footage area of a house increases, its price reliably increases as well.

```python
##1. Data Loading and Exploration
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load the dataset
df = pd.read_csv('homeprices.csv')

# Explore the dataset
print("Dataset Head:\n", df.head())
print("\nDataset Description:\n", df.describe())

# Visualize the data
plt.scatter(df['area'], df['price'], color='blue', marker='o')
plt.title('Home Prices vs. Area')
plt.xlabel('Area (sq ft)')
plt.ylabel('Price (USD)')
plt.show()
```

```
Dataset Head:
    area   price
0  2600  550000
1  3000  565000
2  3200  610000
3  3600  680000
4  4000  725000

Dataset Description:
              area          price
count     5.000000       5.000000
mean   3280.000000  626000.000000
std     540.370243   74949.983322
min    2600.000000  550000.000000
25%    3000.000000  565000.000000
50%    3200.000000  610000.000000
75%    3600.000000  680000.000000
max    4000.000000  725000.000000
```

Step 2: Data Preparation

Before building the model, the data was separated into features (X) and target labels (y). Using Scikit-learn's train_test_split function, the data was divided into an 80/20 split—meaning 80% of the data was

allocated for training the model, and 20% was set aside for testing. This split is standard practice because it ensures the model can be evaluated on unseen data, allowing us to accurately measure how well it generalizes to new inputs rather than just memorizing the data it was trained on.

```python
##2. Data Preparation and Splitting
from sklearn.model_selection import train_test_split

# Define feature (X) and target (y)
X = df[['area']] # Feature must be a 2D DataFrame/array
y = df['price']  # Target is a 1D Series

# Split the data (80% training, 20% testing)
# Note: With only 5 rows, this yields 4 training rows and 1 testing row.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("Training data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
Training data shape: (4, 1)
Testing data shape: (1, 1)
```

Step 3: Model Training

Next, a Linear Regression model was instantiated and fitted to the training data. During this training phase, the algorithm calculated the optimal line of best fit by determining the mathematical relationship between the area and the price. Based on the training data provided, the model calculated a coefficient (slope) of 128.27 and an intercept of 211,542.06.

```python
##3. Building the Model
from sklearn.linear_model import LinearRegression

# Initialize and train the model
model = LinearRegression()
model.fit(X_train, y_train)

# View the mathematical components
print(f"Coefficient (m): {model.coef_[0]}")
print(f"Intercept (b): {model.intercept_}")
```

```
Coefficient (m): 128.27102803738316
Intercept (b): 211542.05607476638
```

```python
##4. Evaluating the Model
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Make predictions on the test set
y_pred = model.predict(X_test)

# Calculate metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred) # Note: R2 cannot be properly calculated with only 1 test sample.

print(f"Mean Absolute Error (MAE): {mae}")
print(f"Mean Squared Error (MSE): {mse}")
print(f"Root Mean Squared Error (RMSE): {rmse}")
print(f"R-squared (R2) Score: {r2}")
```

```
Mean Absolute Error (MAE): 31355.140186915873
Mean Squared Error (MSE): 983144816.1411468
Root Mean Squared Error (RMSE): 31355.140186915873
R-squared (R2) Score: nan
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_regression.py:1266: UndefinedMetricWarning: R^2 score is not well-defined with less than two samples.
  warnings.warn(msg, UndefinedMetricWarning)
```

Step 4: Evaluation

Once the model was trained, its predictive accuracy was tested on 20% of the data unseen. The model's performance was evaluated using standard regression metrics: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and the R^2 score. MAE provides the average absolute magnitude of prediction errors. Conversely, MSE and RMSE square the errors before averaging them. Because of this squaring step, RMSE penalizes larger errors much more heavily than MAE, making it a highly effective metric for identifying whether the model's predictions contain any extreme outliers.

```python
##3. Building the Model
from sklearn.linear_model import LinearRegression

# Initialize and train the model
model = LinearRegression()
model.fit(X_train, y_train)

# View the mathematical components
print(f"Coefficient (m): {model.coef_[0]}")
print(f"Intercept (b): {model.intercept_}")
```

```
Coefficient (m): 128.27102803738316
Intercept (b): 211542.05607476638
```

```python
##4. Evaluating the Model
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Make predictions on the test set
y_pred = model.predict(X_test)

# Calculate metrics
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred) # Note: R2 cannot be properly calculated with only 1 test sample.

print(f"Mean Absolute Error (MAE): {mae}")
print(f"Mean Squared Error (MSE): {mse}")
print(f"Root Mean Squared Error (RMSE): {rmse}")
print(f"R-squared (R2) Score: {r2}")
```

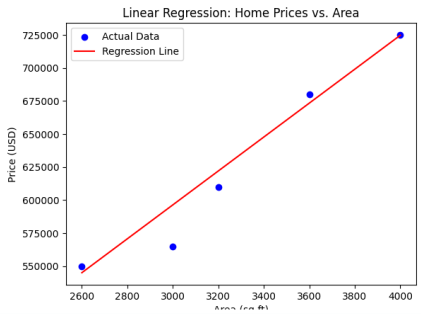
```
Mean Absolute Error (MAE): 31355.140186915873
Mean Squared Error (MSE): 983144816.1411468
Root Mean Squared Error (RMSE): 31355.140186915873
R-squared (R2) Score: nan
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_regression.py:1266: UndefinedMetricWarning: R^2 score is not well-defined with less than two samples.
  warnings.warn(msg, UndefinedMetricWarning)
```

Step 5: Visualization

To visually assess the model's predictive power, the predicted values were plotted alongside the original dataset. In the chart below, the blue data points represent the actual home prices relative to their square footage, while the red line represents the linear regression model's predicted trajectory. The line passes cleanly through the data points, visually confirming the model's ability to capture the dataset's positive correlation.

```python
##5. Visualizing the Results
# Predict across the whole dataset to draw the regression line
y_pred_all = model.predict(X)

plt.scatter(X, y, color='blue', marker='o', label='Actual Data')
plt.plot(X, y_pred_all, color='red', label='Regression Line')
plt.title('Linear Regression: Home Prices vs. Area')
plt.xlabel('Area (sq ft)')
plt.ylabel('Price (USD)')
plt.legend()
plt.show()
```



Terminal

Link to Code:

https://drive.google.com/drive/folders/1fTVtO2kH4Qujz8E_SXug4NG1x0RRoOOK?usp=sharing

Conclusion

Through completing this assignment, I gained practical experience in building a machine learning model from scratch and executing the standard data science pipeline. By exploring the dataset, I learned how to identify and visualize the strong positive correlation between a home's square footage and its market price. Furthermore, training and evaluating the Simple Linear Regression model solidified my understanding of how algorithms calculate the line of best fit and how metrics like RMSE and MAE are used to quantify prediction errors.